

Color Matching Game – Start Screen Upgrade Guide with Code

This document explains clearly how to upgrade the existing Color Matching Game by adding a professional start screen.

All required code snippets are included so you can directly implement them in the lab.

GOAL OF THE IMPROVEMENT

Current Flow:

App opens → Game starts immediately in ContentView

New Required Flow:

App opens → Start Screen → User selects level → Game Screen opens

We will implement this using four main steps.

STEP 1 – CREATE NEW START SCREEN FILE

Create a new SwiftUI file named StartView.swift and add the following code:

```
import SwiftUI
```

```
struct StartView: View {
```

```
    @State private var selectedLevel: Int? = nil
```

```
    var body: some View {
```

```
        NavigationView {  
            VStack(spacing: 30) {
```

```
                Text("Color Matching Game")  
                    .font(.largeTitle)  
                    .bold()
```

```
                Text("Tap a level to start the game")  
                    .font(.headline)  
                    .foregroundColor(.gray)
```

```
                VStack(spacing: 15) {
```

```
                    NavigationLink(destination: ContentView(currentLevel: 1), tag: 1, selection: $selectedLevel) {  
                        LevelCard(title: "3 x 3")  
                    }  
                }
```

```
                    NavigationLink(destination: ContentView(currentLevel: 2), tag: 2, selection: $selectedLevel) {  
                        LevelCard(title: "5 x 5")  
                    }  
                }
```

```
                    NavigationLink(destination: ContentView(currentLevel: 3), tag: 3, selection: $selectedLevel) {  
                        LevelCard(title: "7 x 7")  
                    }  
                }
```

```

        }
        Spacer()
    }
    .padding()
}
}
}
}

```

STEP 2 – CREATE LEVEL CARD COMPONENT

Add this reusable component inside StartView.swift below the previous code:

```

struct LevelCard: View {
    var title: String

    var body: some View {
        Text(title)
            .font(.title2)
            .bold()
            .frame(maxWidth: .infinity)
            .padding()
            .background(Color.blue.opacity(0.2))
            .cornerRadius(10)
    }
}

```

STEP 3 – MODIFY CONTENTVIEW TO ACCEPT LEVEL

Open ContentView.swift and change the currentLevel declaration.

// OLD CODE (REMOVE THIS)

```
@State private var currentLevel = 1
```

// NEW CODE (USE THIS INSTEAD)

```
@State var currentLevel: Int
```

Then add this initializer inside ContentView struct:

```

init(currentLevel: Int) {
    self._currentLevel = State(initialValue: currentLevel)
}

```

STEP 4 – REMOVE LEVEL PICKER FROM CONTENTVIEW

Delete this entire Picker section from ContentView.swift because level is now selected from StartView:

```

Picker("Level", selection: $currentLevel) {
    Text("Level 1").tag(1)
    Text("Level 2").tag(2)
    Text("Level 3").tag(3)
}

```

```
.pickerStyle(SegmentedPickerStyle())  
.padding(.horizontal)  
.onChange(of: currentLevel) {  
    startGame()  
}
```

STEP 5 – CHANGE APP ENTRY POINT

Open Color_Matching_GameApp.swift and replace ContentView with StartView:

```
// OLD  
WindowGroup {  
    ContentView()  
}  
  
// NEW  
WindowGroup {  
    StartView()  
}
```

FINAL RESULT

After completing all these steps:

- The app will first show a welcome screen
- User will see three options: 3x3, 5x5, 7x7
- After tapping a level, game screen will open
- Only the selected level grid will be used
- Game will behave exactly like a real mobile game

WHAT YOU LEARN FROM THIS UPGRADE

- Creating multiple SwiftUI screens
- NavigationView and NavigationLink
- Passing data between views
- Using initializers in SwiftUI
- Better application architecture

Follow this guide step by step in the lab and you will successfully implement the improved version.